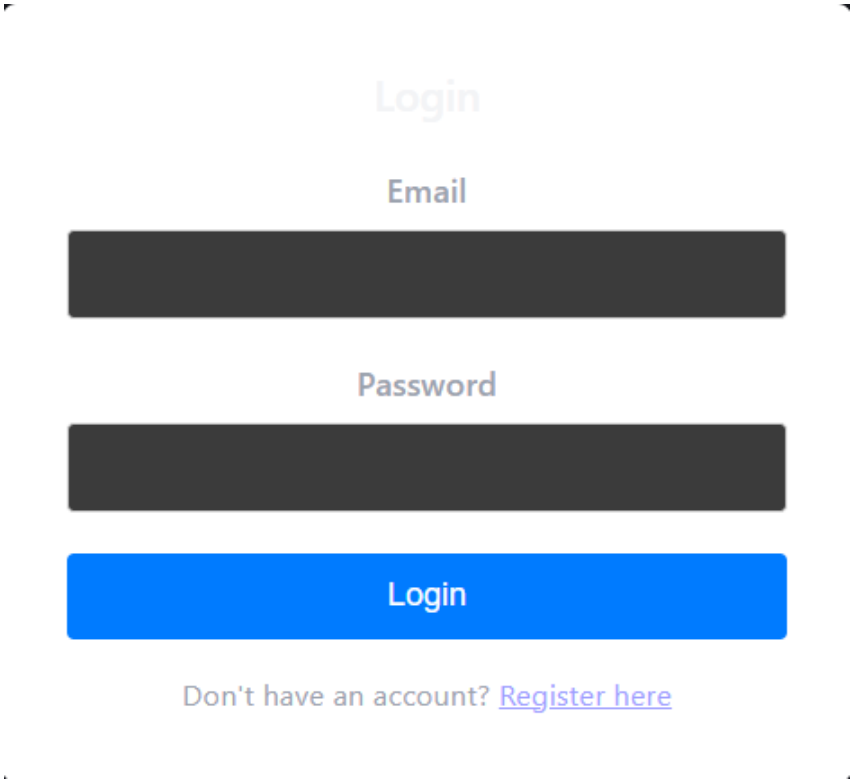
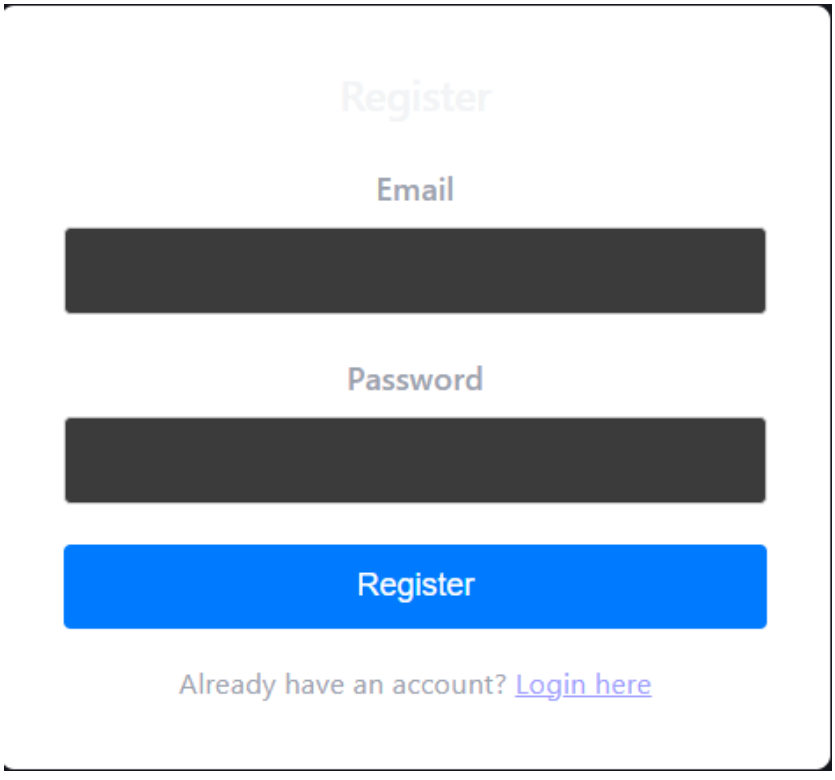
Simon Heart E. Delos Cientos **BSIT-3**

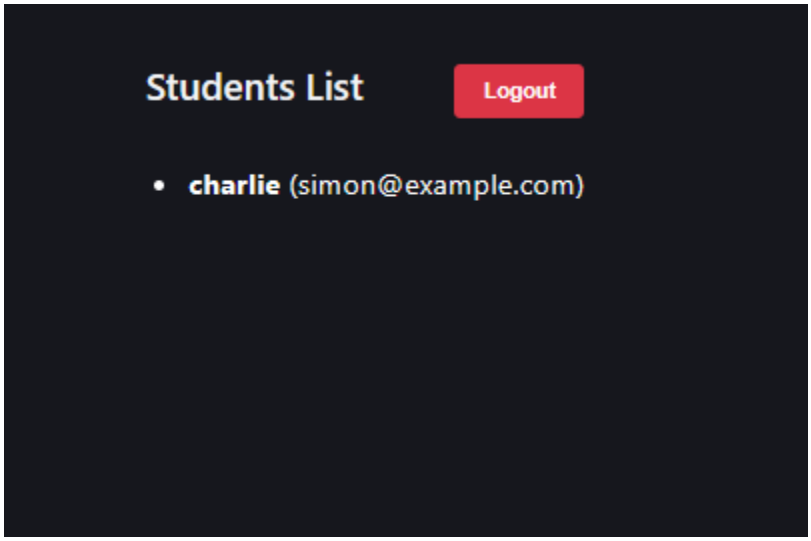
Login page



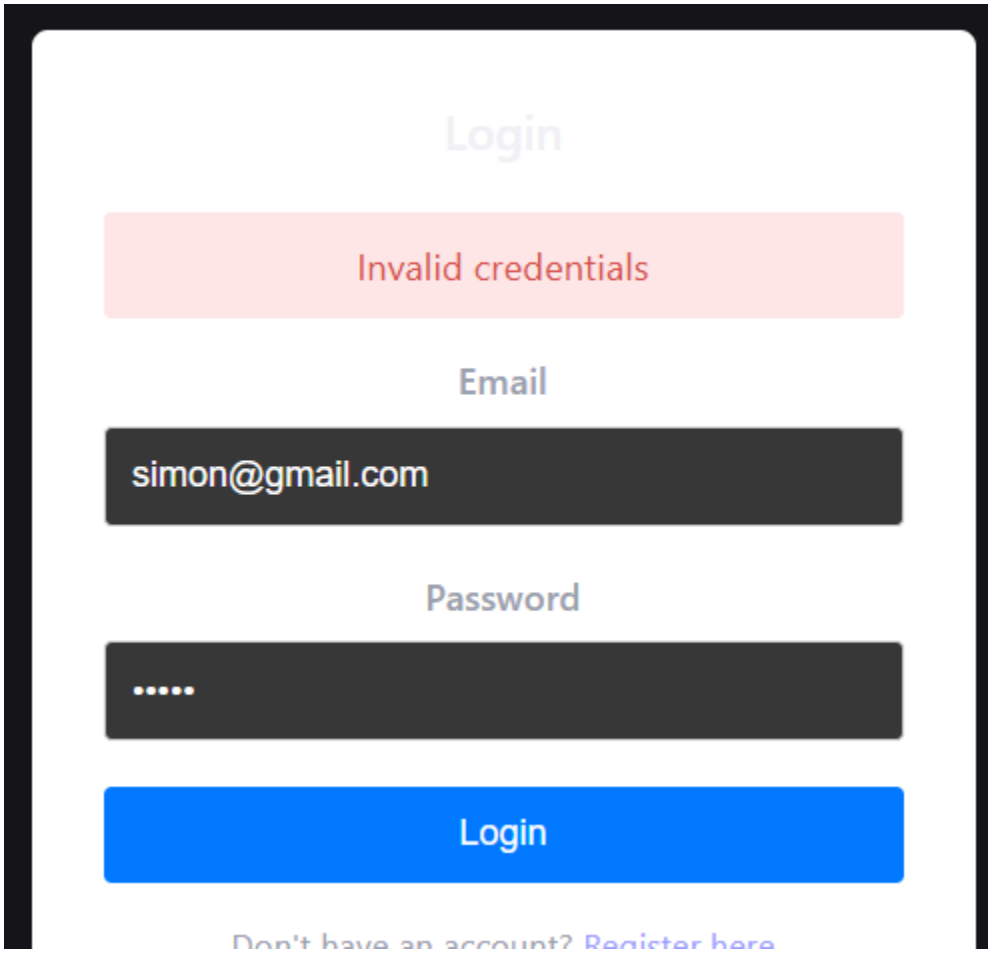
Register page



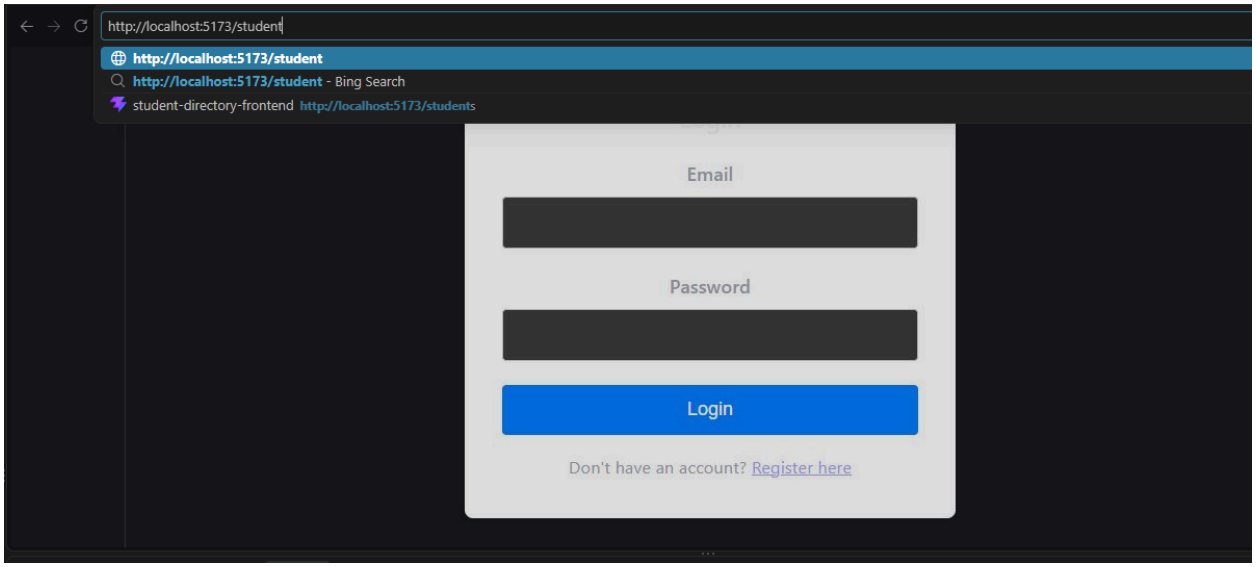
Successful login

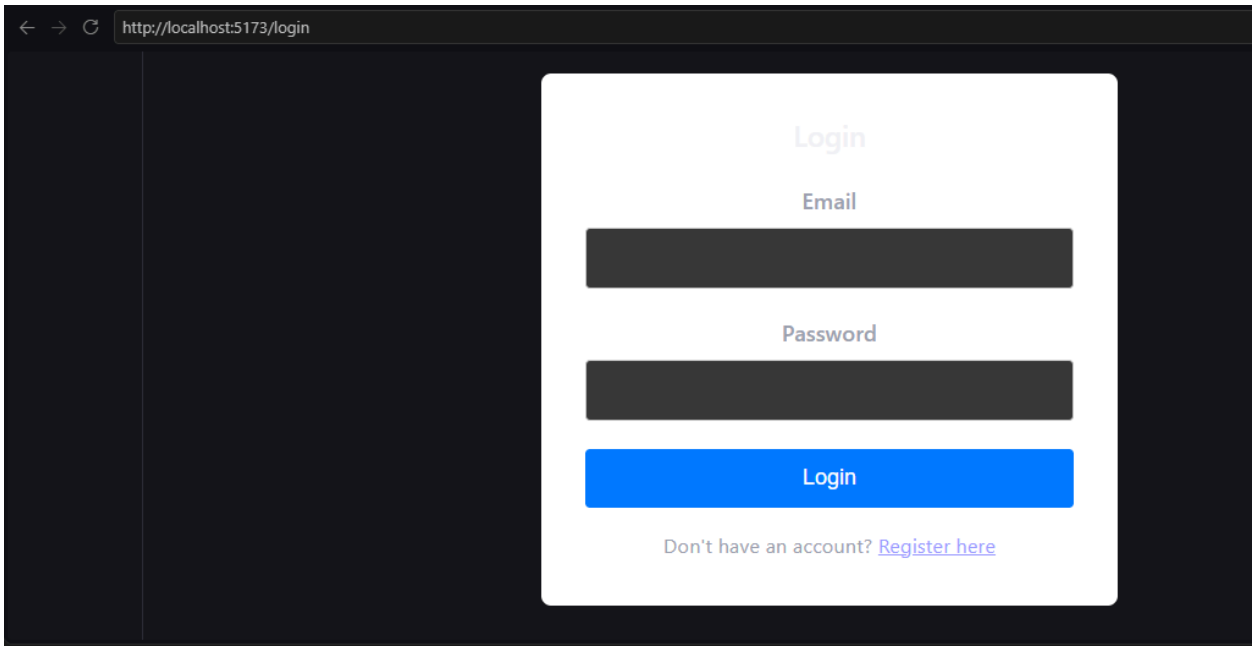


Failed login

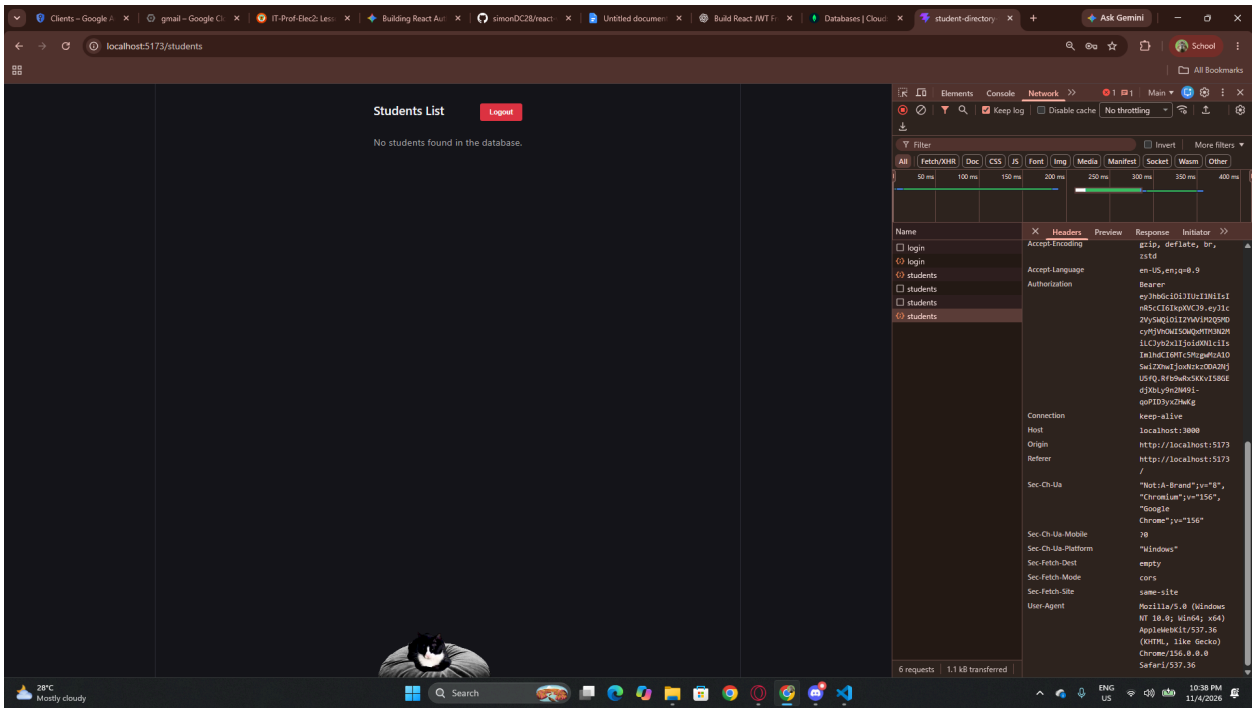


Protected route redirect





Browser DevTools (Network tab)



Reflection Questions

Why do we store the token in localStorage? What is one security risk of doing so, and what is a more secure alternative?

We store the token in localStorage so that the user's login state persists across page reloads and browser tabs without requiring re-authentication. The primary security risk of localStorage is vulnerability to Cross-Site Scripting (XSS) attacks, as any malicious script executing on the domain can access stored tokens. A more secure alternative is storing tokens in an httpOnly cookie with SameSite flags, which prevents client-side JavaScript from accessing the token. Alternatively, applications can store short-lived access tokens strictly in React memory while using an httpOnly cookie for refresh tokens.

What is the purpose of the AuthContext? Why not just pass the token as a prop to each component?

AuthContext provides a centralized global state in React to manage authentication status, user data, and authentication methods (login, logout) across the entire application. Passing the token as a prop to every component leads to "prop drilling," where intermediate components that do not need the token are forced to receive and pass it down. AuthContext allows any deeply nested component to subscribe directly to useAuth() without polluting intermediate component signatures. This keeps the codebase clean, maintainable, and decoupled from component hierarchy layout.

What happens on the frontend when the JWT expires? Describe the user experience you would want to implement.

When a JWT expires, subsequent API requests sent from the frontend fail and return a 401 Unauthorized status code from the server. Without proper handling, the application might fail silently or render empty/broken states while keeping the user visually logged in. To deliver an optimal user experience, the frontend should catch 401 responses globally using an API interceptor or error-handling utility. It should automatically log the user out, clear local storage, redirect them to /login, and display a helpful notification such as "Your session has expired. Please log in again."

What is the difference between the backend's requireAuth middleware and the frontend's ProtectedRoute component? Why do we need both?

The frontend's ProtectedRoute component controls UI presentation by preventing unauthenticated users from seeing protected page layouts and automatically redirecting them to the login screen. In contrast, the backend's requireAuth middleware acts as the true security boundary, cryptographically verifying the JWT signature on every incoming API request before serving data. Frontend route protection is purely for user experience and can easily be bypassed by manually tweaking local storage or sending direct HTTP requests using tools like Postman. We need both because ProtectedRoute creates a smooth UI workflow, while requireAuth secures actual backend data and resources.